High-Performance Request System

Backend Degradation Under Load

Adaptive versus Static Routing

Scenario 4 Benchmark Analysis

Report:	Scenario 4
Focus:	Backend degradation
Strategies:	RR, LC, WRR, Adaptive V1
Workload:	100 req/s open-loop, 60 s measurement
Repetitions:	Five runs per strategy

October 2026

Abstract

This report examines how four routing strategies respond when one backend degrades during an otherwise stable workload. The system uses three backends with initial latencies near 20 ms and routing weights $3 : 1 : 2$. Thirty seconds into a 60-second measurement window, Backend 3 rises to approximately 500 ms, while Backend 1 and Backend 2 remain near their initial latency. The workload is a 100 request-per-second open-loop stream with 32 workers, five runs per strategy, and 6,000 requests per run.

Round robin (RR) and weighted round robin (WRR) produce whole-run mean latencies of approximately 101.8 ms and 101.7 ms, respectively. Least connections (LC) and Adaptive V1 produce approximately 26.5 ms and 26.3 ms. Adaptive therefore reduces the whole-run mean by roughly 74.2% relative to the static policies, but it is only about 0.8% lower than LC. That difference is small relative to run-to-run variation and is not treated as evidence that Adaptive is superior to LC. The important distinction is between static policies that preserve their configured weights and dynamic policies that respond to observed backend behavior.

In a representative Adaptive run, Backend 3 falls from roughly one-third of pre-degradation assignments to approximately 2% after the degradation boundary, while Backend 1 and Backend 2 absorb the redirected traffic. The shift is visible within one measurement second. Static policies continue to send one-third of requests to the degraded backend, so their whole-run means understate the post-degradation steady-state cost of approximately 180 ms. The result demonstrates responsiveness to a latency-visible degradation; it does not establish that Adaptive is generally better than LC.

1 Objective

The objective of Scenario 4 is to determine whether Adaptive V1 changes its routing behavior when a single backend becomes substantially slower while the offered load and the other backends remain stable. The scenario separates two questions that are often conflated:

1. Do static routing policies continue to honor their configured weights when those weights no longer reflect backend health?
2. Does a dynamic policy detect the degradation and redirect traffic quickly enough to protect aggregate latency?

The scenario is therefore a degradation-response test rather than a general load-balancing benchmark. It is also a comparison against a strong dynamic baseline: LC already reacts to in-flight work and should naturally reduce traffic to a slow backend. A credible result must distinguish between "Adaptive is better than static" and "Adaptive is better than LC." This scenario provides evidence for the former but not for the latter.

2 Experimental Configuration

Table 1 summarizes the workload and the degradation schedule. All backends begin near 20 ms. The routing weights are fixed at 3 : 1 : 2 for Backend 1, Backend 2, and Backend 3, respectively. Backend 3 receives one-third of the weighted assignments under WRR and one-third of the assignments under unweighted RR.

Table 1: Scenario 4 configuration.

Parameter	Value	Notes
Backends	3	Backend 1, Backend 2, Backend 3
Initial latency	~ 20 ms	All backends before degradation
Routing weights	3 : 1 : 2	Backend 1 : Backend 2 : Backend 3
Degradation	Backend 3 $\rightarrow \sim 500$ ms	Begins 30 s into measurement
Load	100 req/s	Open-loop request stream
Workers	32	Concurrent request workers
Warm-up	5 s	Excluded from measurement
Measurement	60 s	6,000 requests per run
Repetitions	5 per strategy	20 runs total
Errors	0	No failed requests recorded

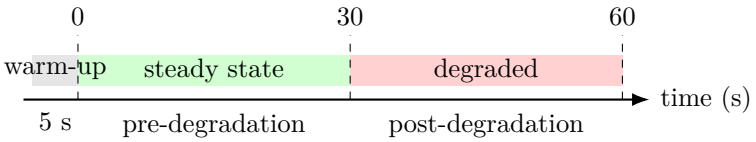


Figure 1: Measurement timeline. The 5-second warm-up is excluded. Backend 3 degrades 30 seconds into the 60-second measurement window and remains slow for the remainder of the run.

The five runs per strategy provide 30,000 measured requests per strategy and 120,000 measured requests overall. The aggregate latency statistics in this report are computed from the five

run-level means rather than from a single pooled request trace. This makes the run-to-run variation visible and avoids presenting one unusually favorable or unfavorable run as the result. Figure 1 shows the measurement schedule.

3 Results

3.1 Aggregate Latency

Table 2 lists the mean latency of each run. The strategies divide into two clear groups. RR and WRR remain near 101 ms, while LC and Adaptive remain near 26 ms. The separation is large enough that it is visible without statistical testing, but the comparison between LC and Adaptive is not.

Table 2: Per-run mean end-to-end latency, in milliseconds.

Strategy	Run 1	Run 2	Run 3	Run 4	Run 5	Median	Mean
RR	101.74	101.70	102.01	101.71	101.96	101.74	101.824
LC	26.52	26.55	26.54	26.52	26.30	26.52	26.486
WRR	101.77	101.87	101.90	102.00	101.13	101.87	101.734
Adaptive	26.05	26.46	26.34	26.75	25.79	26.34	26.278

Table 3 and Figure 2 summarize the same result alongside tail-latency ranges. Adaptive's whole-run mean is 74.2% lower than RR's and 74.2% lower than WRR's. Against LC, the difference is only 0.208 ms, or about 0.8%. The run-level ranges overlap, and no significance test was performed. The appropriate conclusion is that Adaptive and LC perform comparably in this workload, not that Adaptive outperforms LC.

Table 3: Summary of run-level mean latency, tail ranges, and errors.

Strategy	Mean (ms)	Median (ms)	P95 range (ms)	P99 range (ms)	Errors
RR	101.8	101.7	502.2–502.6	502.8–503.1	0/30,000
LC	26.5	26.5	22.5–22.9	23.7–24.2	0/30,000
WRR	101.7	101.9	501.4–502.3	501.8–502.8	0/30,000
Adaptive	26.3	26.3	21.8–22.9	23.9–27.0	0/30,000

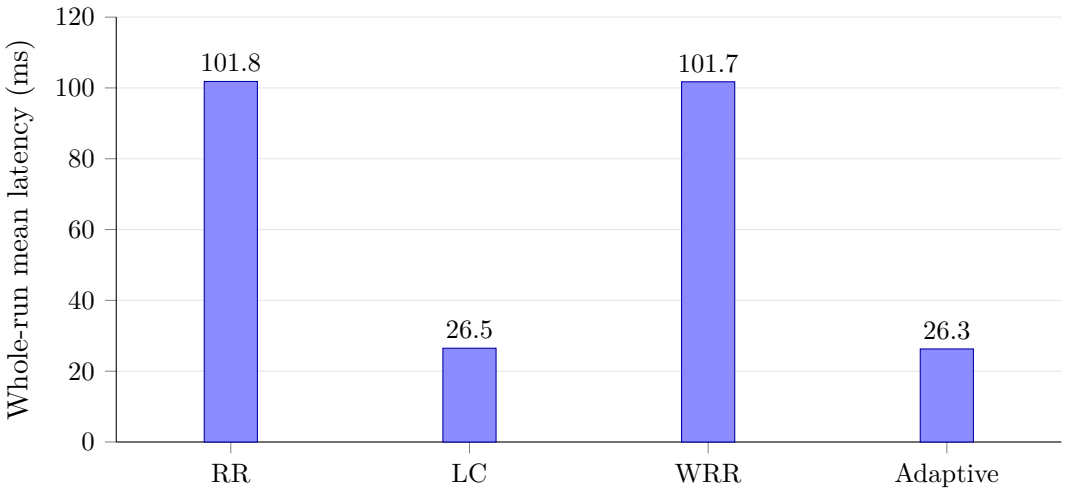


Figure 2: Whole-run mean latency by strategy. Lower is better. The static policies remain near 102 ms; LC and Adaptive remain near 26 ms. Error bars are omitted because the run-level ranges are summarized in Table 3.

The whole-run mean deserves careful interpretation. Each run contains approximately 30 seconds of normal operation and 30 seconds of degradation. For a policy that preserves the $3 : 1 : 2$ weights, the expected latency before degradation is 20 ms, while the expected latency after degradation is

$$\mathbb{E}[L_{\text{post}}] = \frac{3(20) + 1(20) + 2(500)}{6} \approx 180\,\text{ms}. \tag{1}$$

Averaging the two phases gives approximately 100 ms, which is close to the observed whole-run means for RR and WRR. The whole-run average therefore does not mean that static routing is performing well after degradation. It means that the normal phase dilutes the degraded phase. The post-degradation steady-state cost is roughly 180 ms, not 102 ms.

3.2 Tail Latency

Tail latency makes the same distinction more sharply. RR and WRR have P95 and P99 values near 502 ms, which is essentially the latency of the degraded backend. LC and Adaptive have P95 values between approximately 22 ms and 23 ms, and P99 values between approximately 24 ms and 27 ms. Static policies continue to place a substantial share of requests on the slow backend, so their upper tail is dominated by that backend.

Adaptive's P99 range extends to 27.0 ms, slightly above LC's maximum of 24.2 ms. This does not change the main conclusion. The ranges overlap, the difference is small relative to the static-policy gap, and the experiment was not designed to resolve sub-millisecond or low-single-digit-millisecond differences with statistical confidence. The result supports the claim that Adaptive avoids the degraded backend; it does not support a claim of tail latency superiority over LC.

No failed requests were recorded in any strategy. Scenario 4 therefore tests latency degradation rather than hard failure. Error handling, retries, timeouts, and partial availability are outside the scope of this measurement.

3.3 Adaptive Backend Distribution

Table 4 shows the Backend 1, Backend 2, and Backend 3 assignment counts for a representative Adaptive run. Backend 3 receives 1,053 of 6,000 requests over the whole run, or 17.6%. That whole-run share is consistent with a policy that sends about one-third of the pre-degradation requests to Backend 3 and approximately 2% of the post-degradation requests. The arithmetic expectation for those two phases is approximately 17.7%, which closely matches the observed share.

Table 4: Adaptive backend distribution in a representative run.

Backend	Requests	Whole-run share	Observation
Backend 1	2,371	39.5%	Absorbs redirected traffic
Backend 2	2,576	42.9%	Absorbs redirected traffic
Backend 3	1,053	17.6%	Falls to about 2% post-degradation
Total	6,000	100.0%	Representative run

The post-degradation Backend 3 share is small but not zero. A small residual share is not necessarily a defect: some traffic may remain due to selection probabilities, probing behavior, or boundary effects in one-second buckets. The available trace does not establish which explanation applies. What the trace does show is that the residual share is too small to dominate the aggregate latency, and that Backend 1 and Backend 2 receive the displaced load.

The distribution is also a reminder that "Adaptive" does not mean "perfectly equal." The configured weights, the score components, and the observed backend states all influence assignment. In this run, Backend 2 receives more requests than Backend 1 despite Backend 1 having the larger static weight. That is not an error by itself; the relevant question is whether the resulting distribution protects latency and tracks backend health.

3.4 Transition Behavior

Table 5 and Figure 3 show representative one-second assignment counts around the degradation boundary. These are illustrative snapshots, not averages across all runs. In the first trace, Backend 3 receives 33 requests at 29 seconds, 3 requests at 30 seconds, and 2 requests at 31 seconds. Backend 1 and Backend 2 absorb the difference. The second trace shows the same pattern beginning at its 30-second bucket.

Table 5: Representative one-second backend assignment counts around the degradation boundary. Counts are illustrative, not averaged across runs.

Trace	Time	Backend 1	Backend 2	Backend 3	Total
Run A	29 s	33	34	33	100
Run A	30 s	49	47	3	99
Run A	31 s	47	51	2	100
Run B	30 s	64	34	1	99
Run B	31 s	65	33	2	100

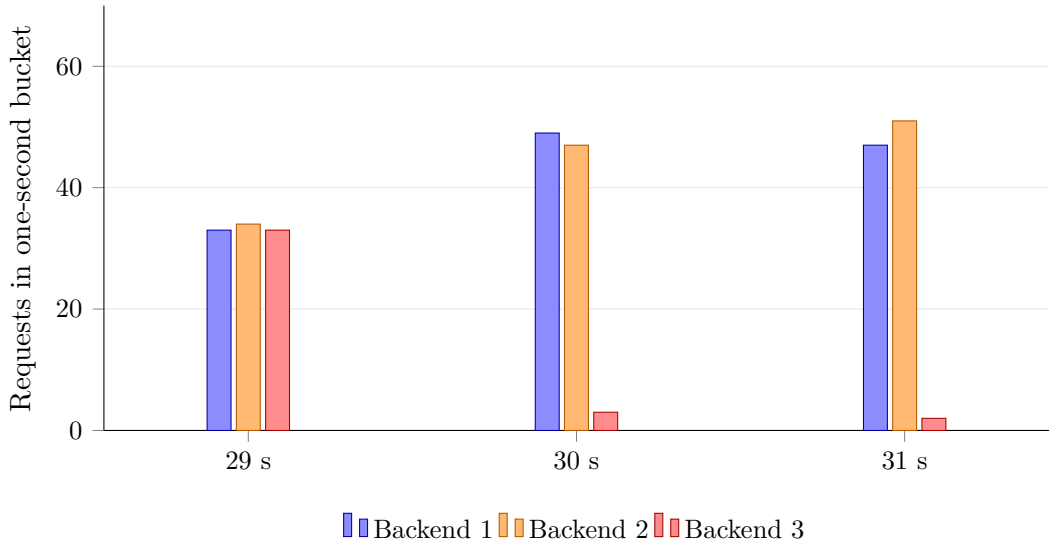


Figure 3: Representative Adaptive transition for Run A. Backend 3 drops from 33 requests at 29 seconds to 3 at 30 seconds and 2 at 31 seconds. The displaced load moves to Backend 1 and Backend 2.

The transition is fast relative to the measurement window. The trace does not prove that the strategy responds within a fixed control interval, because the one-second buckets aggregate many decisions and because the exact instant of the degradation within the bucket is not resolved. It does show that the strategy does not remain pinned to its pre-degradation distribution for multiple seconds.

4 Adaptation Mechanism

Adaptive V1 combines three signals into a scalar score for each backend. The form used in this scenario is

$$S_i(t) = 0.45\,C_i(t) + 0.40\,R_i(t) + 0.15\,U_i(t), \tag{2}$$

where $C_i(t)$ represents normalized active connections, $R_i(t)$ represents normalized observed response time, and $U_i(t)$ represents normalized CPU utilization. Lower scores are preferred. Backends with lower scores are more likely to be selected on subsequent routing decisions.

The important signal in this scenario is response time. Backend 3's latency rises by roughly a factor of 25, from about 20 ms to about 500 ms. That increase is visible to the router through completed-request observations and moves Backend 3's score upward relative to the other backends. The CPU term does not explain the shift: the degradation is a latency injection, not a CPU saturation event. This distinction matters because a CPU-only controller could miss the problem entirely, while a response-time-aware controller sees it directly.

The score is a relative preference, not a health check. It does not by itself prove that a backend is unhealthy, and it does not guarantee that a backend receives zero traffic. It provides a mechanism for shifting assignment when one backend becomes a poor choice relative to the others. The observed distribution is consistent with that mechanism, but the experiment does not isolate the contribution of each score component.

5 Comparative Interpretation

The four strategies answer the degradation question differently.

- **RR** distributes requests evenly and ignores latency. It sends approximately one-third of requests to the degraded backend before and after the degradation.
- **WRR** preserves the configured $3 : 1 : 2$ weights and also sends approximately one-third of requests to the degraded backend. Its result is nearly identical to RR because the degraded backend's share is the same in both policies.
- **LC** reacts indirectly through in-flight work. A slower backend accumulates more outstanding requests, so it receives fewer new assignments. This produces a large latency improvement without using an explicit response-time score.
- **Adaptive V1** reacts through a multi-signal score. In this scenario, the response-time signal moves the assignment away from the degraded backend and produces latency comparable to LC.

The comparison against LC is the most important guard against overclaiming. LC is simple, already dynamic, and performs essentially as well as Adaptive in this workload. Adaptive's 74.2% advantage is relative to RR and WRR, not relative to LC. The experiment therefore validates degradation responsiveness, not the necessity or superiority of a multi-signal adaptive policy.

That does not make the result uninteresting. A multi-signal policy can, in principle, incorporate information that LC cannot see directly, such as CPU pressure or other backend-local signals. Scenario 4 shows that the policy can respond when response time provides a clear signal. It does not show that the additional signals improve outcomes in every workload; that requires separate scenarios with different bottlenecks.

6 Why Static Policies Fail Under Degradation

Static policies encode a capacity plan. They do not encode current health. When Backend 3 degrades, RR and WRR continue to honor their assignment rules, so the degraded backend keeps receiving the same share of traffic. The result is not a subtle tail effect; it is a structural mismatch between the routing rule and the system state.

For WRR, the expected latency under preserved weights is given by Equation 1. Before degradation, every backend is near 20 ms, so the weighted mean is also near 20 ms. After degradation, the 2/6 share assigned to Backend 3 dominates the weighted mean and pushes the expected value to approximately 180 ms. The whole-run mean of about 102 ms is the average of the healthy and degraded phases, not a representation of steady-state behavior after degradation.

RR has the same structural problem with a different assignment rule. It sends one-third of requests to each backend. After degradation, that one-third share is sufficient to produce a whole-run mean near 102 ms and a post-phase mean near 180 ms. WRR's configured weights do not help because the weight that matters after degradation is the weight assigned to the unhealthy backend.

The static policies are not "wrong" in the sense of violating their specification. They are behaving exactly as configured. The failure is that their specification does not include backend health. A static policy can be appropriate when backend capacity and health are stable, but it is fragile when a backend's service time changes independently of its configured weight.

7 Why Adaptive Routing Succeeds

Adaptive routing succeeds in this scenario because the degradation is visible in a signal the router observes. Backend 3's response time rises sharply, its score rises, and its assignment share falls. The transition table shows the effect at the one-second granularity: 33 assignments at 29 seconds, 3 at 30 seconds, and 2 at 31 seconds in the representative trace. The displaced load moves to the other backends, which are still near their initial latency.

The mechanism is a feedback loop, and feedback loops require care. A router that reacts too aggressively can oscillate between backends, especially when the underlying signal is noisy or when the degraded backend recovers and degrades repeatedly. This scenario does not test flapping, recovery, or hysteresis. It shows a clean, sustained degradation with a clear signal, which is the easiest case for a feedback controller.

The residual Backend 3 share of about 2% is also part of the picture. It shows that the policy is not a hard circuit breaker. The router continues to observe and may continue to assign a small amount of traffic. That behavior can be useful for recovery detection, but it also means that a small amount of latency risk remains. The experiment does not determine whether the residual share is intentional exploration, a selection-probability artifact, or a boundary effect.

LC achieves a similar outcome through a different mechanism. Its in-flight signal is an indirect but effective proxy for backend service time. The fact that LC performs as well as Adaptive is a useful reminder that a simple dynamic policy can be sufficient when the relevant state is captured by outstanding work. Adaptive's case would be stronger if it demonstrated a workload in which LC's proxy is insufficient but the multi-signal score is not.

8 Limitations

This scenario has several boundaries that limit generalization.

- **Single degradation profile.** Backend 3 jumps to approximately 500 ms once and stays there. The experiment does not cover gradual degradation, partial recovery, repeated flapping, or multiple simultaneous degradations.
- **Small number of runs.** Five runs per strategy are enough to show a large separation from the static policies, but not enough to make confident claims about small differences. No significance test was performed.
- **Whole-run aggregates.** The run-level means average healthy and degraded phases. The post-degradation steady-state is inferred from the weights and the measured transition behavior, not reported as a separate aggregate across all runs.
- **Representative transition data.** The one-second snapshots in Table 5 and Figure 3 are illustrative. They show that the shift occurs, but they are not averages over all runs.
- **Open-loop workload.** The generator offers 100 req/s with 32 workers and does not test overload, queue growth, or client-side backpressure.
- **No hard failures.** All requests succeeded. Timeouts, retries, connection resets, and partial availability are not exercised.
- **No production network variability.** The experiment does not include wide-area latency variation, packet loss, or heterogeneous client behavior.
- **No claim against LC.** Adaptive's advantage over LC is within the noise visible in this

experiment. The result does not justify replacing LC with a more complex policy on latency grounds alone.

- **Limited signal isolation.** The score combines CPU, response time, and in-flight pressure. The experiment does not isolate the contribution of each term or test a CPU-bound degradation.

The earlier CPU-pressure scenario (Scenario 3) is relevant context: when CPU pressure was the manipulated variable, it did not produce a comparable routing shift because the signal was not strong or sustained enough. Scenario 4 shows the complementary case: when response time clearly changes, the controller responds. The two results together suggest that the choice of signal matters more than the presence of an "adaptive" label.

9 Engineering Interpretation

The first engineering lesson is that static weights are not a health mechanism. They are useful for encoding known capacity differences, but they must be combined with health-aware assignment if backend service time can change independently of the weights. A weighted policy that continues to send one-third of traffic to a backend operating at 500 ms is not preserving fairness; it is preserving a stale plan.

The second lesson is that response time is a practical leading signal for this class of degradation. It is observed directly by the router, it reflects the user-visible effect, and it moves quickly when a backend becomes slow. CPU utilization is not a substitute: a latency injection can leave CPU unchanged, and a CPU-only controller can therefore remain blind to the problem.

The third lesson is that LC is a strong baseline. It reacts to the consequence of slowness through in-flight work, without an explicit multi-signal model. Any adaptive policy that claims superiority over LC must show a workload in which LC's proxy is insufficient, not merely a workload in which both policies beat static routing. Scenario 4 does not provide that evidence.

The fourth lesson is that feedback control needs damping and observability. A production implementation should expose per-backend assignment shares, score components, and transition behavior. It should include hysteresis or rate-limiting to avoid oscillation under noisy signals, and it should retain a safe residual path for recovery detection. Those mechanisms are not tested here, but the one-second transition data show why they matter: the controller can move a large share of traffic quickly, and the same responsiveness could be destabilizing if the signal were noisy.

Finally, the result should be read as a capability demonstration. Adaptive V1 can detect a latency-visible degradation and redirect traffic. It is comparable to LC in this workload. It is not a proven general replacement for LC, and it is not evidence that multi-signal routing is always worth its complexity. Figure 4 summarizes the causal interpretation.

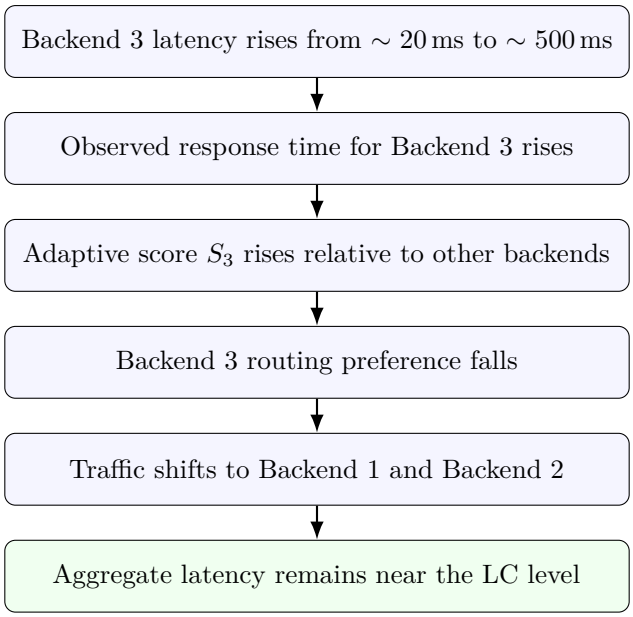


Figure 4: Causal interpretation of the observed Adaptive behavior. The response-time signal is the primary driver in this scenario; the other score components are not isolated by the experiment.

10 Conclusion

Scenario 4 shows a clear separation between static and dynamic routing under a sustained backend degradation. RR and WRR preserve their configured shares and continue to send roughly one-third of requests to a backend operating at approximately 500 ms. Their whole-run means are near 102 ms, and their post-degradation steady-state cost is approximately 180 ms. LC and Adaptive avoid the degraded backend and keep whole-run means near 26 ms.

Adaptive's whole-run mean is approximately 74.2% lower than the static policies and approximately 0.8% lower than LC. The latter difference is small, overlapping across runs, and not supported by a significance test. The correct interpretation is that Adaptive responds effectively to this degradation and performs comparably to LC, not that it is generally superior to LC.

The representative transition trace shows Backend 3 falling from 33 assignments at 29 seconds to 3 at 30 seconds and 2 at 31 seconds, with Backend 1 and Backend 2 absorbing the load. The whole-run Backend 3 share of 17.6% is consistent with roughly one-third of assignments before degradation and approximately 2% afterward. Static policies cannot produce this shift because their weights do not depend on backend health.

The result is a capability demonstration with clear limitations. It does not test flapping, recovery, multiple degradation severities, hard failures, or overload. It does not establish that a multi-signal policy is necessary when LC already performs well. The next step is to test whether Adaptive provides a measurable advantage in workloads where LC's in-flight proxy is insufficient, and to add formal statistical comparison and stability analysis before making production claims.